

timeweb>cloud

Влюбляем в облачные технологии

ваш гид в мир железок, игр и IT



256K+

Охват за 30 дней

Timeweb Cloud

То самое облако

1 594,92 Рейтинг

30 787 Подписчики

[Подписаться](#)

j4niwzis

22 часа назад

Объяснение лямбда исчисления через Python

🔗 Сложный 🕒 9 мин 👁 7.8K

Блог компании Timeweb Cloud, Функциональное программирование*, Python*, Программирование*, Ненормальное програ

[Обзор](#)

» Введение

Реализация универсальной рекурсии в одну строку и объяснение ключевых конструкций лямбда-исчисления на Python. Подойдёт даже тем, кто не очень знаком с Python. Если хотите понять, как из одних лишь функций строятся булевы, списки и числа и, быть может, попробовать до реализации некоторых алгоритмов самостоятельно — добро пожаловать под кат.

Начну с небольшой личной истории. Чисто из академического любопытства захотелось реализовать универсальную рекурсию в одну строку на Python, не прибегая к использованию именованных функций, на лямбдах. Мне было известно про то, что это возможно в лямбда исчислении, хоть мне и был неизвестен способ. После нескольких минут размышлений у меня получилось реализовать это, а процесс решения таких задач понравился, из этого вышли поиски другого в лямбда исчислении. Так и появилась идея о написании этой статьи. Моё решение оказалось аналогом Y-комбинатора для Python.

Сразу обозначу, что перевод синтаксиса лямбда исчисления в синтаксис Python не означает полную эквивалентность этого кода с языком Python, хотя в некоторых случаях результат будет тот же.

» Принципы работы лямбда исчисления

В лямбда исчислении есть всего несколько фундаментальных операций: аппликация и абстракция.

Абстракция или λ -абстракция, в свою очередь, строит функции по заданным выражениям. Можно задать функцию как `( $\lambda x$ .expr)`, где x — имя переменной, а `expr` выражение, которое выйдет после применения функции. В `expr` можно использовать x . Если перевести это на Python, то будет `(lambda x: expr)`.

Аппликация означает вызов функции. Обычно обозначается как $f\ a$, где “ f ” это функция, а “ a ” — аргумент. Это соответствует математической записи $f(a)$. Такая же запись используется и в Python. `(lambda x: x)(y)` будет как `(( $\lambda x$ .x) y)`. Аргументами вполне могут служить и другие функции. А функции могут возвращать другие функции.

Функции от нескольких аргументов в лямбда исчислении записываются как `( $\lambda x\ y$ .expr)`. Эта запись будет эквивалентна записи `( $\lambda x$ . $\lambda y$ .expr)`. Если переводить на синтаксис Python, то будет `(lambda x: lambda y: expr)`. Т.е. функции от множества аргументов задаются как функции, принимающие N -й аргумент, возвращающие функции, которые принимают $(N + 1)$ -й элемент и лишь в конце выражение, которое может использовать эти аргументы.

Описанный процесс превращения функций многих переменных в функцию одной переменной называется карринг (также: каррирование).

В этом есть некоторое отличие лямбда исчисления от лямбд в Python. В Python записи `(lambda x, y: x(y))` и `(lambda x: lambda y: x(y))` не эквивалентны. `(lambda x, y: x(y))` сразу требует несколько аргументов (вызов через (x, y)), когда как `(lambda x: lambda y: x(y))` требует цепочки вызовов (вызов через $(x)(y)$)

Запись `f a b c` эквивалентна записи `((f a) b) c)`

Стратегии выполнения

Лямбда исчисление можно выполнять по разному. f задаётся как `( $\lambda x$ .ax)( $\lambda y$ .by) c)` или, если при переводе на синтаксис Python, `(lambda x: a(x))((lambda y: b(y))(c))`.

Можно различным образом раскрывать f : сначала раскрывать функцию (ленивый подход), или перед этим раскрывать её аргументы (энергичный подход).

Раскрытия с ленивым подходом: `(λx .a x)(λy .b y) c) => a((λy .b y) c) => a(b c)
(lambda x: a(x))((lambda y: b(y))(c)) => a((lambda y: b(y))(c)) => a(b(c))`

Раскрытия с энергичным подходом: `(λx .a x)(λy .b y) c) => (λx .a x)(b c) => a (b c)
(lambda x: a(x))((lambda y: b(y))(c)) => (lambda x: a(x))(b(c)) => a(b(c))`

Несложно заметить, что результат этих раскрытий получился одинаковый. Это доказывается теоремой Чёрча — Россера. Но стоит понимать, что теорема говорит лишь о том, что самая упрощенная форма будет одинаковой для любого порядка раскрытия, но это не означает достижимость этой формы при любом порядке раскрытия. К примеру, `(lambda x, y: x)(42, (lambda x: x(x))(lambda x: x(x)))` будет выполняться вечно при энергичном подходе, но достигнет конечного результата при ленивом.

Если рассматривать конкретно Python, то исполнение у него энергичное.

› Различные алгоритмы на лямбда исчислении

Сразу оговорюсь, что тут функции будут использовать функции от множества аргументов в стиле Python. И написано оно для языка Python, поэтому стоит учитывать энергичное выполнение. Но всё это можно переписать и в обычном лямбда исчислении.

Рекурсия

Многие вычисления могут требовать длительное (или даже бесконечное) вычисление, длительность которого зависит от аргументов.

Для этого в некоторых языках есть циклы, рекурсия. Рассмотрим рекурсивную реализацию факториала.

```
def fact(arg):  
    return 1 if arg == 0 else arg * fact(arg - 1)
```

[Объяснить с](#)  SourceCraft

Внутри функции факториала используется вызов себя же через имя, так получается рекурсия. Но данный подход не получится сделать напрямую в лямбда исчислении, так как в лямбда исчислении у функций нет имён. Но это достижимо через некоторые функции, которые можно описать на лямбда исчислении. Они передают в функцию себя же в каком-то виде.

Предлагаю попробовать это реализовать своими силами в коде Python. Дам немного тестов

```
f = () # Ваша функция, задавайте через лямбды и не используйте прямую рекурсию через имя  
  
assert(f(lambda s, y: y * s(y-1) if y != 0 else 1)(0) == 1)  
assert(f(lambda s, y: y * s(y-1) if y != 0 else 1)(1) == 1)  
assert(f(lambda s, y: y * s(y-1) if y != 0 else 1)(2) == 2)  
assert(f(lambda s, y: y * s(y-1) if y != 0 else 1)(3) == 6)  
assert(f(lambda s, y: y + s(y-1) if y != 0 else 0)(2) == 3)  
assert(f(lambda s, y: y)(42) == 42)
```

[Объяснить с](#) 

Как видно из тестов, используется подход Python к определению функций нескольких аргументов. И, так как используется энергичный подход вычисления, следует это учитывать при написании кода.

► Решение

Это решение будет аналогично Z-комбинатору, который является аналогом Y-комбинатора энергичного выполнения.

Именно такое решение у меня получилось из той личной истории, только потом оказалось, что оно уже изобретено в подобном виде и до меня :)

Y комбинатор

Y комбинатор был введён известным американским учёным Хаскеллом Карри. Он имеет следующий вид

$$Y = \lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$$

[Объяснить с](#) 

Нетрудно заметить, что он уйдёт в бесконечную рекурсию при использовании энергичного выполнения. Для решения этой проблемы был придуман Z комбинатор. В нём следующий аргумент определён явно, что отложит вычисления.

```
Z = λf.(λx.f (λv.x x v)) (λx.f (λv.x x v))
```

[Объяснить с 🧐](#)

Моё решение в прошлой части по поведению аналогично этому Z комбинатору.

Это всё комбинаторы неподвижной точки.

Boolean значения

В чистом лямбда-исчислении нет отдельных типов — всё выражается функциями. Булевы значения удобно закодировать как функции-матчеры: каждое значение — функция, которая принимает два аргумента (ветку истина и ветку ложь) и вызывает нужную.

```
true = lambda x, y: x
false = lambda x, y: y
```

[Объяснить с 🧐](#)

Такая кодировка естественно служит для pattern matching, т.к. значение содержит логику выбора ветки.

Можно определить операции с такими булевыми значениями

```
# Негация
not_ = lambda b: b(false, true)

# И
and_ = lambda a, b: a(b, false)

# Или
or_ = lambda a, b: a(true, b)

# Эквивалентность
eq = lambda a, b: a(b, not_(b))
```

[Объяснить с 🧐](#)

Кодировка Скотта

Структура — это функция, которая принимает обработчики для каждого конструктора (например, on_empty и on_pair для списка) и вызывает нужный. Это фактически отложенный pattern-matching.

```
nil = lambda on_empty, on_pair: on_empty
cons = lambda first, second: lambda on_empty, on_pair: on_pair(first, second)
```

[Объяснить с 🧐](#)

Пример создания списка 0 -> 1 -> nil:

```
my_list = cons(0, cons(1, nil))
# my_list = lambda on_empty, on_pair: on_pair(0, lambda on_empty, on_pair: on_pair(1, 1, on_empty, on_pair))
```

[Объяснить с](#) 

Из примечательного тут то, что на каждом шаге принимаются функции для разбора, создавая тем самым вложенную структуру. Это позволяет менять функции при разборе.

Разбор (pattern-matching):

```
list_get_first = lambda l: l(None, lambda x, xs: x)
list_is_empty = lambda l: l(True, lambda _, __: False)
```

[Объяснить с](#) 

Эту кодировку можно использовать не только для списков, но и для других структур.

К примеру, так можно сделать числа:

```
fix = (lambda g: (lambda x: x(x))(lambda s: lambda arg: g(s(s), arg)))

# Ноль (Zero)
# Просто возвращает первый аргумент
zero = lambda on_zero, on_succ: on_zero

# Следующее число (Successor)
# Принимает число 'n' и возвращает функцию,
# которая при вызове отдаст этот 'n' во второй аргумент.
# Это позволяет увеличить уровень вложенности на 1, что и означает увеличение числа на 1
succ = lambda n: lambda on_zero, on_succ: on_succ(n)

# Примеры чисел
one = succ(zero)
two = succ(one)
three = succ(two)

# Получает обычное число в Python из такого
to_python_int = fix(lambda s, num: num(0, lambda x: 1 + s(x)))
```

[Объяснить с](#) 

Прибавление на единицу задаётся как succ(x), где x - число, к которому прибавляют.

Реализацию вычитания я предлагаю попробовать реализовать читателю. Вот тесты

```
assert(to_python_int(pred(one)) == 0)
assert(to_python_int(pred(two)) == 1)
assert(to_python_int(pred(three)) == 2)
```

[Объяснить с](#) 

Кодировка Чёрча

Эта кодировка была придумана Алонзо Чёрчем. Вместо того, чтобы создавать вложенную структуру, которая принимает функции на каждом шаге, он задал структуры как свёртку.

```
my_list = lambda f, x: f(0, f(1, x))
```

[Объяснить с](#) 

Стоит заметить, что функция используется одна и та же. Это задаёт некие ограничения с точки зрения гибкости использования.

Получение первого элемента списка выглядит так:

```
list_get_first = lambda l: l((lambda x, xs: x), None)
```

[Объяснить с](#) 

В этой же кодировке можно задавать и числа.

Эти числа — это просто применение функции N раз: 0 это x, 1 это f(x), 2 это f(f(x)) и так далее.

```
zero = lambda f, x: x
one = lambda f, x: f(x)
two = lambda f, x: f(f(x))
three = lambda f, x: f(f(f(x)))

# Как можно заметить, в такой реализации не потребовался комбинатор неподвижной точки,
to_python_int = lambda n: n(lambda x: x + 1, 0)

succ = lambda num: lambda f, x: f(num(f, x))
```

[Объяснить с](#) 

Если операция прибавления единицы задаётся легко, то вычитания единицы уже нет. Предлагаю попробовать решить эту задачу. Тут будет несколько решений и подсказки для них. Советую посмотреть все решения

Тесты

```
assert(to_python_int(pred(one)) == 0)
assert(to_python_int(pred(two)) == 1)
assert(to_python_int(pred(three)) == 2)
```

[Объяснить с](#) 

Метод Стивена Клини

▸ [Подсказка](#)

▸ [Решение](#)

Метод Барендрегта

▸ [Подсказка](#)

▸ [Решение](#)

Моё решение 1

▸ [Подсказка](#)

▸ [Решение](#)

Моё решение 2

Его у меня получилось придумать в ходе написания этой статьи. Оно получилось достаточно элегантным и его можно обобщить на другие алгоритмы (вычитание, деление и т.д.) и другие типы данных в кодировке Чёрча.

Его можно найти [тут](#).

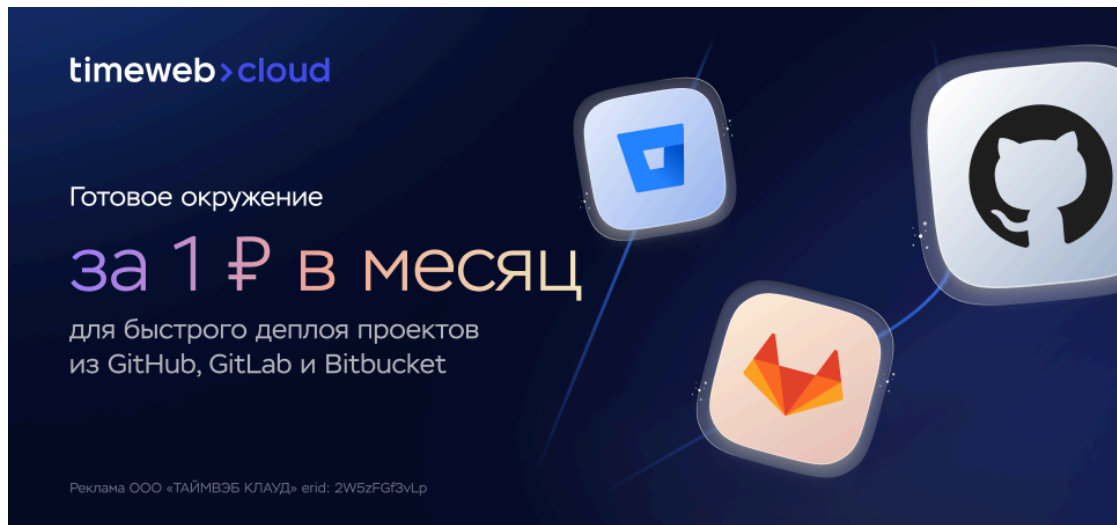
Хочу написать научную статью с этим алгоритмом, но опыта научных публикаций не имею. Ищу опытных соавторов.

» Заключение

Лямбда-исчисление показывает, что из одних лишь функций и их применения можно выразить очень многое: управление потоком, рекурсию, булевы значения, списки, числа.

В статье мы прошли путь от базовых понятий (абстракция и аппликация) через различие стратегий редукции до практических кодировок — Чёрча и Скотта — и показали, как это всё реализуется на Python, который является энергичным языком.

Новости, обзоры продуктов и конкурсы от команды Timeweb.Cloud — в нашем Telegram-канале ↩



Теги: python, рекурсия, функциональное программирование, абстракция, комбинаторы, лямбда исчисление, редукция, timeweb_статьи, аппликация

Хабы: Блог компании Timeweb Cloud, Функциональное программирование, Python, Программирование, Ненормальное программирование



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



256K+

Охват за 30 дней

Timeweb Cloud

То самое облако



8K+

Охват за 30 дней

11

Карма

@j4niwzis

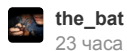
Пользователь

Подписаться



Комментарии 2

Публикации



the_bat

23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники

Простой 4 мин 16K

+97

31

27



interpres

18 часов назад

Электроинструмент становится хуже, и это делается намеренно

Простой 11 мин 27K

Обзор

Перевод

+73

47

87



prohronus

19 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере

5 мин 9.5K

Кейс

+54

9

9



alizar

22 часа назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

Простой 7 мин 12K

Обзор

+36

62

18



AlexSerbul

23 часа назад

Программирование с AI-ассистентом — похороните меня под плинтусом

Средний 14 мин 10K

Мнение

+28

33

12



TrexSelectel

19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки

5 мин 8.9K

Обзор

+27

10

0

mariklolik

19 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

Средний

7 мин

7.6K

Кейс

+23

23

6

nick_dyuba

23 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и TіріTір

5 мин

7.2K

Recovery Mode

+23

8

5

infgotoinf

11 часов назад

Perl — зря забытый язык программирования?

11 мин

8.8K

Из песочницы

+16

13

10

SLY_G

23 часа назад

Что такое «мышечная память» и можно ли её развить?

4 мин

7.2K

Перевод

+16

10

7

Показать еще

ИНФОРМАЦИЯ

Сайт	timeweb.cloud
Дата регистрации	11 августа 2011
Дата основания	9 февраля 2022
Численность	201–500 человек

Местоположение

Россия

Представитель

[Timeweb Cloud](#)

ССЫЛКИ

[Серверы под любые задачи](#)

[timeweb.cloud](#)

[Облачные базы данных \(DBaaS\)](#)

[timeweb.cloud](#)

[Объектное S3-хранилище](#)

[timeweb.cloud](#)

[Серверы 152-ФЗ](#)

[timeweb.cloud](#)

[Kubernetes](#)

[timeweb.cloud](#)

[VDI: Аренда виртуальных рабочих мест](#)

[timeweb.cloud](#)

[Партнерская программа](#)

[timeweb.cloud](#)

[Timeweb News — актуальн](#)

[t.me](#)

[«Релиз в пятницу» — подкаст от команды Т](#)

[www.youtube.com](#)

ВКОНТАКТЕ



Timeweb Cloud

42,854 followers

[Follow on VK](#)

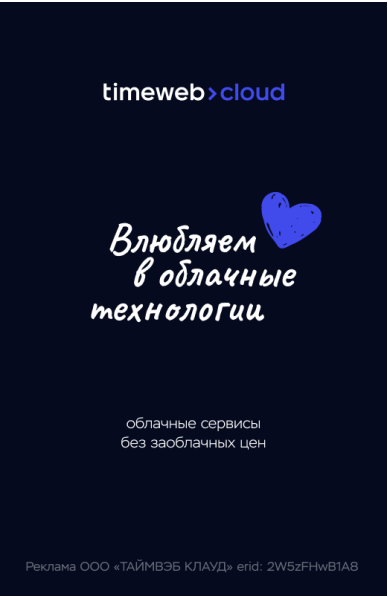
ПРИЛОЖЕНИЯ



Timeweb Cloud в App Store и Google Play

Встречайте наше мобильное приложение для Айфонов и Андроидов. Через него можно полноценно управлять инфраструктурой со смартфона — делать все, что и на десктопе. Скачивайте по ссылке ниже ↓

[Android](#) [iOS](#)



БЛОГ НА ХАБРЕ

22 часа назад

Объяснение лямбда исчисления через Python

7.8K 2

19 апр в 19:05

Китайский шедевр сумрачного гения из 2000-х

13K 7

19 апр в 16:05

«Фабрика монстров» Джорджа Лукаса: как в «Звездных войнах» создавали инопланетян без компьютерной графики

11K 2

18 апр в 22:05

Opus 4.7, Codex управляет ПК, TTS от Google, бренд кроссовок «переобулся» в ИИ-компанию

8.4K 5

17 апр в 21:05

Первая редакция D&D: от хаоса к структуре (и обратно)

6.6K 0

Ваш аккаунт

Войти

Разделы

Статьи

Информация

Устройство сайта

Услуги

Корпоративный блог

Регистрация

Новости

Для авторов

Медийная реклама

Хабы

Для компаний

Нативные проекты

Компании

Документы

Образовательные программы

Авторы

Соглашение

Стартапам

Песочница

Конфиденциальность



Настройка языка

Техническая поддержка

© 2006–2026, Habr